those features and drivers that your embedded application is going to use. Avoid kernel modules. The kernel must have the support of the *initrd* (CONFIG_BLK_DEV_INITRD). Have a look at the maximum allowed size for RAM disks (BLK_DEV_RAM_SIZE). You may need to increase it.

Preparing the disk of your embedded computer

You should know whether your embedded application needs to write to the disk. For example, my application has to write some user preference files from time to time. If so, you need two primary partitions on the disk. The first one is bootable and contains some booting files, the kernel, the *initrd*, and possibly some data that you put outside the *initrd*. If the embedded application has no data stored outside of the *initrd*, there is no need to mount the file system located on the first partition. If needed, your application has to mount this file system as a read only one. If your application does not plan to write any data on the disk, one partition on the disk is sufficient.

The second (possibly smaller) partition and its file system contain data the embedded application can rewrite. Again, mount the file system as read-only. Just before you are going to write something, remount it as a read-write file system. Having finished the writing, remount it as a read-only file system again.

The method proposed above is a very rudimentary protection of the file systems on the embedded disk. The embedded application is capable of running even if the second file system is damaged. Of course, you should protect the embedded system from sudden power offs while writing to the file system, if possible.

Linking your embedded application

Try to link your embedded application with static libraries (*lib*.a*), not with shared libraries (*lib*.so*). You can do it if you write your application as just one (possibly multi-thread) process. Linking the application with the static libraries provides a smaller memory and disk footprint of the embedded system, because only the really used library functions are added.

The static linking is ineffective or impossible if you have to use functions such as *system()*, *popen()*, *fork()*, and so on. You'll have to copy some shared libraries into your *initrd*. The *ldd* command will tell you which ones.

I had started with a statically linked embedded application but later developments forced me to use the shared libraries.

Choosing a bootloader

This depends on the file system you plan to use for your embedded disk. As I have *ext* file systems on my embedded disk, I've chosen the *extlinux* branch of the *syslinux* package by Peter Anvin (*www.syslinux.org*). It supports several file systems including *ext2*, *ext3* and *ext4*. Read the documentation and install it. If you don't like looking at Peter Anvin's copyright sign during the boot process, you can display a nice boot picture of your own.

Creating and populating your *initrd*

There are some special tools for making an *initrd*, but they are not useful here. Create a file of appropriate size, as follows:

```
dd if=/dev/zero of=initrd.img bs=$SIZE count=1
```

…where *$SIZE* should be just big enough to accommodate all files and directories of the *initrd* file system. Let's convert *initrd.img* into a file system, as follows:

```
mke2fs -F -m 0 -N 100 initrd.img
```

and mount it to a local directory:

```
mkdir ./mnt; mount -t ext2 -o loop initrd.img ./mnt
```

Now, you can make the necessary directories using commands like:

```
mkdir ./mnt/dev
```

You'll probably need other directories. This would depend on the organisation of your application and on the other programs your application is going to run. If you plan to mount your embedded disk partitions, add mount points. Use the *mknod* command to make device nodes for devices your application needs; for example:

```
mknod ./mnt/dev/sda  b 8 0
mknod ./mnt/dev/ram0 b 1 0
```

Here, */dev/ram0* is the root device. Copy your embedded application (*$PROGRAM*) as '*linuxrc*' to the *initrd:*

```
cp -p $PROGRAM ./mnt/linuxrc
```

If you use shared libraries and/or other programs, copy them, too. Finally, unmount the new *initrd*:

```
umount ./mnt
```

Now, you have got your *initrd* containing all the necessary items in the *initrd.img* file.

Populating your embedded disk

Let us suppose that the embedded disk has the device node */dev/sda* and */dev/sda1* is its first and bootable partition. Create a file system on the partition and mount it, as follows:

```
mke2fs -m 0 -N 200 /dev/sda1
```